BOOKLY · SOLUTIONS ENGINEERING TAKE-HOME

The model should never decide what's true.

A customer support agent for Bookly, built so that language understanding and business authority are deliberately different parts of the system.

Seyfullah Ural · September 2026

Python · Anthropic API · no agent framework

How a question becomes an answer

Four components, and only the first one is probabilistic.

PROMPTS

System prompt

Tone and judgement only.
No business rules.

ORCHESTRATION

Agent loop

Hand-written, ~60 lines.
Iteration cap, full trace.

TOOLS

Tool layer

Identity checks and refund preconditions live here.

MEMORY

Split state

Transcript for the model.
Verified facts for the app.

THE LOAD-BEARING PART

`check_return_eligibility` is ordinary Python. No model, no prompt, no temperature. It is the only authority on whether something can be returned — and the agent is not permitted to contradict it.

The core knows nothing about HTTP. The web chat and the CLI are both thin adapters over one run_turn(). A voice channel would be a sibling of those files, not a rewrite of anything underneath.

Three decisions worth defending

Each one moves authority out of the model and into code.

01

Eligibility is code, not a prompt

Refund rules live in policy.py and run as plain Python.

TRADED AWAY

Every policy change is a deploy, not a prompt edit.

WORTH IT BECAUSE

It becomes auditable and unit-testable. There is no prompt to jailbreak, and no temperature at which 45 becomes less than 30.

02

The write tool has preconditions

issue_refund refuses unless identity was verified, eligibility passed, and the customer said yes.

TRADED AWAY

More code than a line of prompt instruction.

WORTH IT BECAUSE

"Always confirm first" is a suggestion. This is a precondition. The model is now allowed to be wrong without the customer paying for it.

03

Required arguments generate the dialogue

lookup_order cannot be called without both an order id and an email.

TRADED AWAY

Less freedom in how the agent phrases its way there.

WORTH IT BECAUSE

Nobody wrote a script for collecting them. The schema makes a half-informed call impossible, so the agent asks. Flow control by type signature.

Where the architecture shows up

Three moments from the demo, and one experiment.

IT SAYS NO, AND MEANS IT

A refund request on an order delivered 45 days ago. The agent checks, refuses, explains why, and offers a human. It cannot be argued out of the number 30.

IT ASKS INSTEAD OF GUESSING

"I want to return my book" on an order containing two books. It names both and asks which — rather than picking one and being wrong half the time.

THE EXPERIMENT WORTH RUNNING

Delete the API key and the agent falls back to a scripted, rule-based planner. It is far worse at understanding English — and every single guardrail still holds. Identity is still checked, the 30-day rule still bites, refunds still need confirmation. Because none of that was ever in the prompt.

13 unit tests cover the policy engine and the refund gate — including four ways a persuasive customer might talk the model into a refund it must not make.

What I'd do differently

In order. The first one is not close.

1	**Build the eval set**	The policy half is tested. The model half is not measured at all — did it pick the right tool, did it ask when it should have asked, did it stay grounded? Without that, the next prompt change is a guess.
2	**Persist the session**	State is in-process today. A real deployment needs it in Redis or Postgres, keyed by conversation, so a dropped connection is not a lost customer.
3	**Instrument cost and latency per turn**	Support economics are per-contact. If you cannot see what a resolution costs, you cannot argue for the deflection rate.
4	**Add tone regression tests**	An LLM-as-judge over recorded transcripts, so a prompt tweak that makes the agent subtly colder gets caught before a customer feels it.

The thesis is falsifiable: if the guardrails were really in the architecture and not the prompt, swapping the model out shouldn't change what the system permits. It doesn't.